

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель С.В. Умбетов

«___» _____ 2026 г.

Отчет по лабораторной работе №1
по дисциплине «Устройство и функционирование
интеллектуальных информационных систем»
«Лабораторная работа с табличными
данными в Pandas»

ЛР 09.02.08.19.002

Студент группы	<u>1ИИС-41</u>	<u>Н.Ю. Потапов</u>
	группа	инициалы., фамилия
Преподаватель	<u>старший преподаватель, доцент</u>	<u>С.В. Умбетов</u>
	должность, ученая степень	инициалы., фамилия

Барнаул 2026

Цель работы: освоить базовые и продвинутые приемы работы с табличными данными в библиотеке Pandas на примере набора данных о пассажирах Титаника, научиться загружать, исследовать, фильтровать, агрегировать и преобразовывать данные, а также делать аналитические выводы.

Задачи к работе:

1. Изучить объекты Series и DataFrame библиотеки Pandas.
2. Освоить загрузку данных из CSV-файлов и получение первичной информации о датафрейме.
3. Научиться осуществлять индексацию, доступ к данным и фильтрацию по условиям.
4. Освоить статистические методы, группировку данных и построение сводных таблиц.
5. Научиться обрабатывать пропущенные значения и создавать новые признаки.
6. Провести комплексный анализ датасета Titanic и сформулировать выводы.

Задание принял:



Подпись

Потапов Н.Ю.

ФИО

Ход работы:

Задание 1. Загрузка и первичный осмотр данных.

Ход выполнения: импортировал библиотеку `pandas`, загрузил датасет `train.csv` в `DataFrame` `titanic` с помощью `read_csv`, вывел первые 10 и последние 5 строк методами `head(10)` и `tail()`.

ЗАДАНИЕ 1

Чтобы изменить содержимое ячейки, дважды нажмите на нее (или выберите "Ввод")

```
import pandas as pd  
titanic = pd.read_csv('train.csv')  
display(titanic.head(10))  
print(f"\n\n{{60*'='}}\n\n")  
display(titanic.tail())
```

	PassengerId	Survived	PClass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C

_

	PassengerId	Survived	PClass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W/C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

Рисунок 1. Загрузка датасета и вывод первых и последних строк таблицы.

Задание 2. Получение информации о датафрейме.

Ход выполнения: применил метод `info()` для просмотра типов данных и количества пропусков, `describe()` для описательной статистики числовых признаков и `describe(include='object')` для текстовых столбцов.

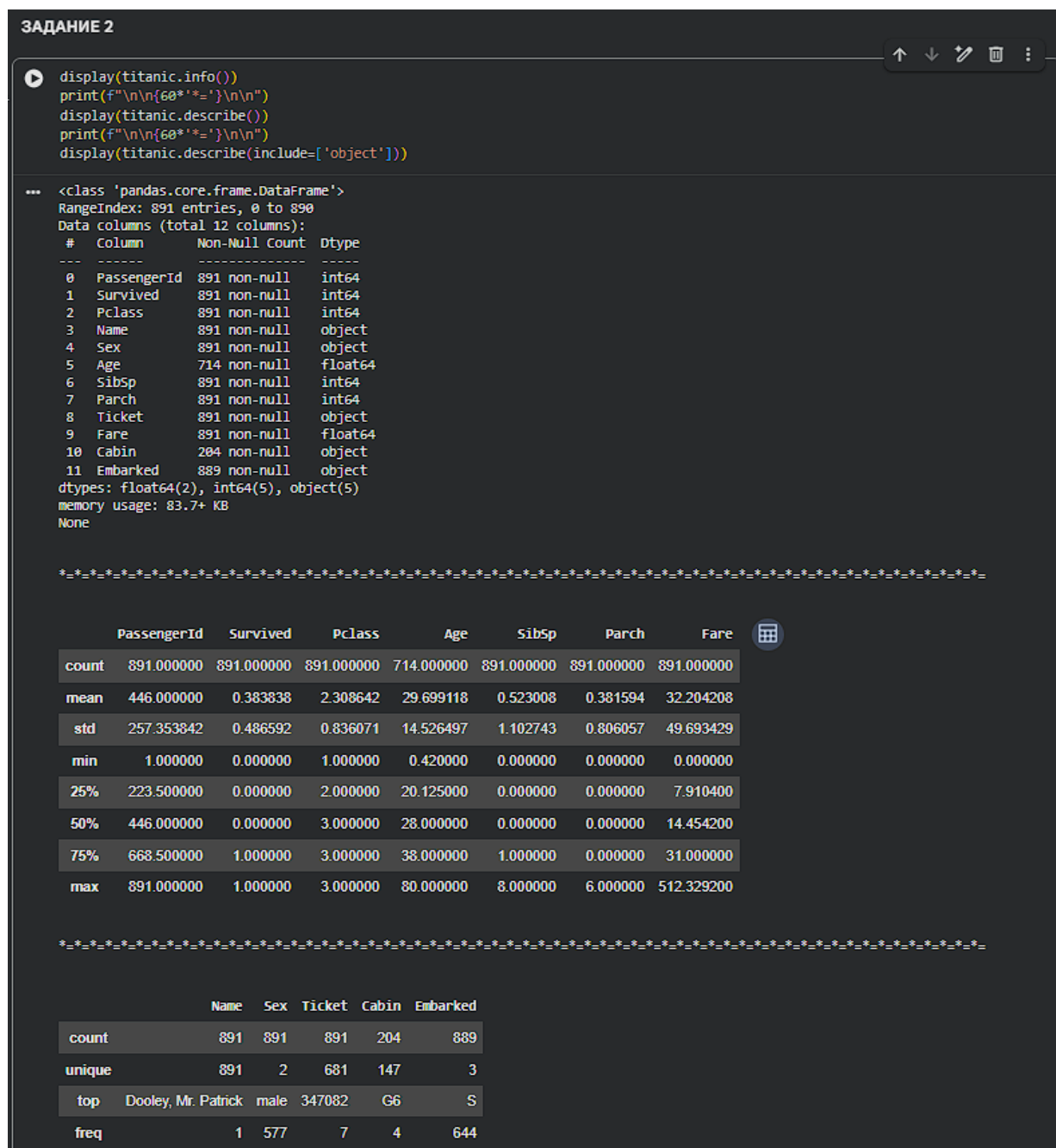
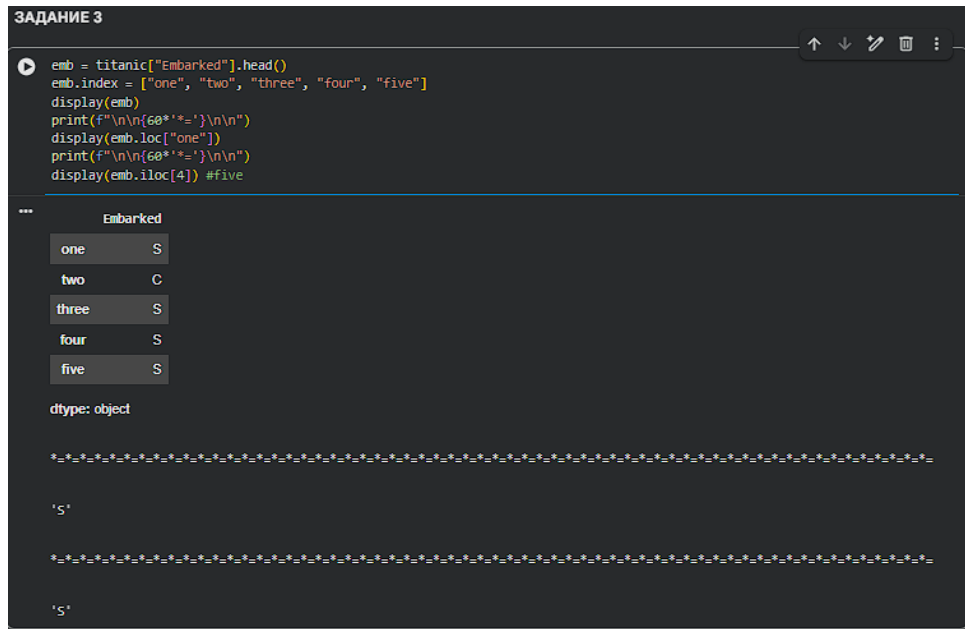


Рисунок 2. Информация о типах данных и статистические характеристики датафрейма.

Задание 3. Работа с Series.

Ход выполнения: извлек столбец Embarked, задал произвольный текстовый индекс для первых 5 элементов и получил доступ к данным через методы loc и iloc.



```
emb = titanic["Embarked"].head()
emb.index = ["one", "two", "three", "four", "five"]
display(emb)
print(f"\n\n{60*'='}\n\n")
display(emb.loc["one"])
print(f"\n\n{60*'='}\n\n")
display(emb.iloc[4]) #five
```

Embarked	
one	S
two	C
three	S
four	S
five	S

dtype: object

```
=====

'S'

=====

'S'
```

Рисунок 3. Создание Series с пользовательским индексом и доступ к элементам.

Задание 4. Статистические методы.

Ход выполнения: вычислил средний возраст пассажиров (mean), минимальную и максимальную стоимость билета (min, max), а также стандартное отклонение возраста (std).



```
age_mean = titanic.Age.mean()
print(f"Средний возраст пассажиров: {age_mean:.2f}")
print(f"\n\n{60*'='}\n\n")
fare_min = titanic.Fare.min()
fare_max = titanic.Fare.max()
print(f"Максимальная цена билета: {fare_max:.2f}\nМинимальная цена билета: {fare_min}")
print(f"\n\n{60*'='}\n\n")
age_std = titanic.Age.std()
print(f"Среднее отклонение возраста {age_std:.2f}")
```

Средний возраст пассажиров: 29.70

```
=====

Максимальная цена билета: 512.33
Минимальная цена билета: 0.0

=====

Среднее отклонение возраста 14.53
```

Рисунок 4. Вычисление базовых статистических параметров столбцов.

Задание 5. Фильтрация данных по условиям.

Ход выполнения: отфильтровал пассажиров старше 30 лет, со стоимостью билета выше средней, а также женщин первого класса, для которых отдельно вычислил средний возраст.

```
ЗАДАНИЕ 5

age_sort = titanic[titanic.Age > 30]
display(age_sort)
print(f"\n\n{60*' '}")
fare_sort = titanic[titanic.Fare > titanic.Fare.mean()]
print(f"Средняя цена билета: {titanic.Fare.mean():.2f}")
display(fare_sort)
print(f"\n\n{60*' '}")
rich_woman = titanic[(titanic.Pclass == 1) & (titanic.Sex == "female")]
display(rich_woman)
age_rw = rich_woman.Age.mean()
print(f"Средний возраст женщин в первом классе: {age_rw:.2f}")
```

Рисунок 5. Код задания

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	2	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
6	7	0	McCarthy, Mr. Timothy J	male	54.0	0	0	17483	51.8825	E48	S
11	12	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	20.5500	C103	S
...	...	...	...	...	...	...	...	...	...	...	...
873	874	0	Vander Cruyssen, Mr. Victor	male	47.0	0	0	345785	9.0000	NaN	S
879	880	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11787	83.1583	C50	C
881	882	0	Markun, Mr. Johann	male	33.0	0	0	349257	7.8958	NaN	S
885	886	0	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382852	20.1250	NaN	Q
890	891	0	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

305 rows x 12 columns

Рисунок 6. Датасфрейм отсортирован по возрасту пассажиров

Средняя цена билета: 32.20											
PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	2	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
6	7	0	McCarthy, Mr. Timothy J	male	54.0	0	0	17483	51.8825	E48	S
23	24	1	Sloper, Mr. William Thompson	male	28.0	0	0	113788	35.5000	A6	S
27	28	0	Fortune, Mr. Charles Alexander	male	19.0	3	2	19950	263.0000	C23 C25 C27	S
...	...	...	...	...	...	...	...	...	...	...	...
856	857	1	Wick, Mrs. George Dennick (Mary Hitchcock)	female	45.0	1	1	36928	104.8867	NaN	S
863	864	0	Sage, Miss. Dorothy Edith "Dolly"	female	NaN	8	2	CA. 2343	69.5500	NaN	S
867	868	0	Roebeling, Mr. Washington Augustus II	male	31.0	0	0	PC 17590	50.4958	A24	S
871	872	1	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	female	47.0	1	1	11751	52.5542	D35	S
879	880	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11787	83.1583	C50	C

211 rows x 12 columns

Рисунок 7. Датасет отсортирован по стоимости билета

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	2	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17509	71.2833	C85	C
3	4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
11	12	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.5500	C103	S
31	32	1	Spencer, Mrs. William Augustus (Marie Eugenie)	female	NaN	1	0	PC 17569	146.5208	B78	C
52	53	1	Harper, Mrs. Henry Sleeper (Myna Haxtun)	female	49.0	1	0	PC 17572	76.7292	D33	C
...	...	...	...	...	...	...	...	...	...	...	...
856	857	1	Wick, Mrs. George Dennick (Mary Hitchcock)	female	45.0	1	1	36928	164.8667	NaN	S
862	863	1	Swift, Mrs. Frederick Joel (Margaret Welles Ba...	female	48.0	0	0	17466	25.9292	D17	S
871	872	1	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	female	47.0	1	1	11751	52.5542	D35	S
879	880	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11767	83.1583	C50	C
887	888	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
94 rows x 12 columns											
Средний возраст женщин в первом классе: 34.61											

Рисунок 8. Сортировка по нескольким признакам: полу пассажира и классу билета

Задание 6. Комбинированная фильтрация и статистика.

Ход выполнения: отобрал выживших пассажиров моложе 18 лет и вычислил для этой группы среднюю стоимость билета и общее количество человек.

ЗАДАНИЕ 6											
<pre> survived_people = titanic[(titanic.Survived == 1) & (titanic.Age < 18)] display(survived_people) sp_fare_mean = survived_people.Fare.mean() sp_count = survived_people.PassengerId.count() print(f"Средняя стоимость билета выживших мл. 18 = {sp_fare_mean:.2f}\nкол-во таких пассажиров: {sp_count:.2f}") </pre>											
PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
9	10	1	Nasser, Mrs. Nicholas (Adele Achem)	female	14.00	1	0	237736	30.0708	NaN	C
10	11	1	Sandstrom, Miss. Marguerite Rut	female	4.00	1	1	PP 9549	16.7000	G6	S
22	23	1	McGowan, Miss. Anna "Annie"	female	15.00	0	0	330923	8.0292	NaN	Q
39	40	1	Nicola-Yarred, Miss. Jamila	female	14.00	1	0	2651	11.2417	NaN	C
43	44	1	Laroche, Miss. Simone Marie Anne Andree	female	3.00	1	2	SC/Paris 2123	41.5792	NaN	C
...	...	...	...	...	...	...	...	...	...	...	...
830	831	1	Yasbeck, Mrs. Antoni (Setini Alexander)	female	15.00	1	0	2659	14.4542	NaN	C
831	832	1	Richards, Master. George Sibley	male	0.83	1	1	29106	18.7500	NaN	S
853	854	1	Lines, Miss. Mary Conover	female	16.00	0	1	PC 17592	39.4000	D28	S
869	870	1	Johnson, Master. Harold Theodor	male	4.00	1	1	347742	11.1333	NaN	S
875	876	1	Najib, Miss. Adele Kiamie "Jane"	female	15.00	0	0	2667	7.2250	NaN	C
61 rows x 12 columns											
Средняя стоимость билета выживших мл. 18 = 34.86											
кол-во таких пассажиров: 61.00											

Рисунок 9. Фильтрация по комбинированному условию и расчет статистики группы.

Задание 7. Группировка данных.

Ход выполнения: сгруппировал данные по классу билета (Pclass), вычислил средний возраст, количество пассажиров и среднюю стоимость билета, отсортировал результат по стоимости по убыванию.

ЗАДАНИЕ 7

```
class_g = titanic.groupby('Pclass')
apfagg_g = class_g.agg(
    Mean_Age = ("Age", "mean"),
    Passenger_Count = ("PassengerId", "count"),
    Fare_Mean = ("Fare", "mean")
)
sort_g = apfagg_g.sort_values(by="Fare_Mean", ascending=True)
display(sort_g)
```

...

	Mean_Age	Passenger_Count	Fare_Mean
Pclass			
3	25.140620	491	13.675550
2	29.877630	184	20.662183
1	38.233441	216	84.154687

Далее: [New interactive sheet](#)

Рисунок 10. Группировка данных с помощью `groupby` и агрегация признаков.

Задание 8. Сводная таблица.

Ход выполнения: построил сводную таблицу `pivot_table` для анализа доли выживших в зависимости от класса билета (строки) и пола (столбцы).

Наибольшая доля выживших оказалась среди женщин первого класса (около 96,8%), а наименьшая — среди мужчин третьего класса (около 13,5%).

ЗАДАНИЕ 8

```
titanic_table = titanic.pivot_table(values='Survived',
    index='Pclass',
    columns='Sex')
display(titanic_table)
```

...

	female	male
Pclass		
1	0.968085	0.368852
2	0.921053	0.157407
3	0.500000	0.135447

Далее: [New interactive sheet](#)

Рисунок 11. Сводная таблица доли выживших по классу и полу.

Задание 9. Работа с пропущенными значениями.

Ход выполнения: оценил количество и процент пропусков, заполнил пропуски в Embarked самым частым значением (S), в Age - средним значением, проверил отсутствие дубликатов по PassengerId.

ЗАДАНИЕ 9

```
null_counts = titanic.isnull().sum()
null_percentages = (titanic.isnull().sum() / len(titanic) * 100).round(2)
sum_table = pd.DataFrame({
    'Null_Count': null_counts,
    'Null_%': null_percentages
})
display(sum_table)
print(f"\n\n{60*'='}\n\n")
display(titanic.Embarked.value_counts())
titanic['Embarked'] = titanic['Embarked'].fillna('S')
titanic['Age'] = titanic['Age'].fillna(titanic['Age'].mean())
print(f"\n\n{60*'='}\n\n")
passg_dup = titanic.duplicated(subset=['PassengerId']).sum()
print(f"Количество дубликатов в PassengerId: {passg_dup}")

#Я хз зачем это делать если дубликатов нет, но раз в задании сказано удалить то я это делаю
titanic = titanic.drop_duplicates(subset=['PassengerId'], keep='first')
```

	Null_Count	Null_%
PassengerId	0	0.0
Survived	0	0.0
Pclass	0	0.0
Name	0	0.0
Sex	0	0.0
Age	0	0.0
SibSp	0	0.0
Parch	0	0.0
Ticket	0	0.0
Fare	0	0.0
Cabin	687	77.1
Embarked	0	0.0

Embarked	count
S	646
C	168
Q	77

dtype: int64

Количество дубликатов в PassengerId: 0

Рисунок 12. Анализ и заполнение пропущенных значений в столбцах.

Задание 10. Создание новых признаков и расширенный анализ.

Ход выполнения: создал признаки FamilySize, IsAlone и AgeGroup (с помощью `pd.cut`), построил сводную таблицу для анализа выживаемости и стоимости билета по возрастным группам.

Дети (до 12 лет) выживали чаще всего (около 58%), тогда как взрослые и пожилые пассажиры имели значительно меньшие шансы на спасение.

```

titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1
titanic['IsAlone'] = titanic['FamilySize'].apply(lambda x: 1 if x == 1 else 0)
titanic['AgeGroup'] = pd.cut(titanic['Age'],
    bins=[0, 12, 18, 35, 60, 100],
    labels=['ребёнок', 'подросток', 'молодой', 'взрослый',
            'пожилой'])
display(titanic[['FamilySize', 'IsAlone', 'Age', 'AgeGroup']])
print(f"\n\n{60*' '}")
age_group = titanic.pivot_table(
    values=['Survived', 'Fare', 'PassengerId'],
    index='AgeGroup',
    aggfunc={"Survived": "mean", "Fare": "mean", "PassengerId": "count"})
display(age_group)

```

	FamilySize	IsAlone	Age	AgeGroup
0	2	0	22.000000	молодой
1	2	0	38.000000	взрослый
2	1	1	26.000000	молодой
3	2	0	35.000000	молодой
4	1	1	35.000000	молодой
...	...	...	...	...
886	1	1	27.000000	молодой
887	1	1	19.000000	молодой
888	4	0	29.699118	молодой
889	1	1	26.000000	молодой
890	1	1	32.000000	молодой

891 rows x 4 columns

FutureWarning: The default value of observed=False is deprecated

```
age_group = titanic.pivot_table(
    values=['Survived', 'Fare', 'PassengerId'],
    index='AgeGroup',
    columns=['Fare', 'PassengerId', 'Survived'],
    aggfunc=["mean", "count", "mean"],
    observed=True)
```

AgeGroup	Fare	PassengerId	Survived
ребёнок	31.536352	69	0.579710
подросток	33.451313	70	0.428571
молодой	27.356166	535	0.353271
взрослый	44.259858	195	0.400000
пожилой	41.371214	22	0.227273

Рисунок 13. Создание новых признаков и сводная таблица по возрастным группам.

Задание 11. Комплексный аналитический отчет.

Ход выполнения: сгруппировал данные по порту посадки (Embarked), вычислил средний возраст, среднюю стоимость билета, долю выживших и долю пассажиров первого класса для каждого порта.

Порт С (Шербур) показал наибольшую долю выживших (55,3%) и самую высокую среднюю стоимость билета, что связано с большим количеством пассажиров первого класса. Порт S (Саутгемптон) принял больше всего пассажиров, но доля выживших там была самой низкой (33,9%).

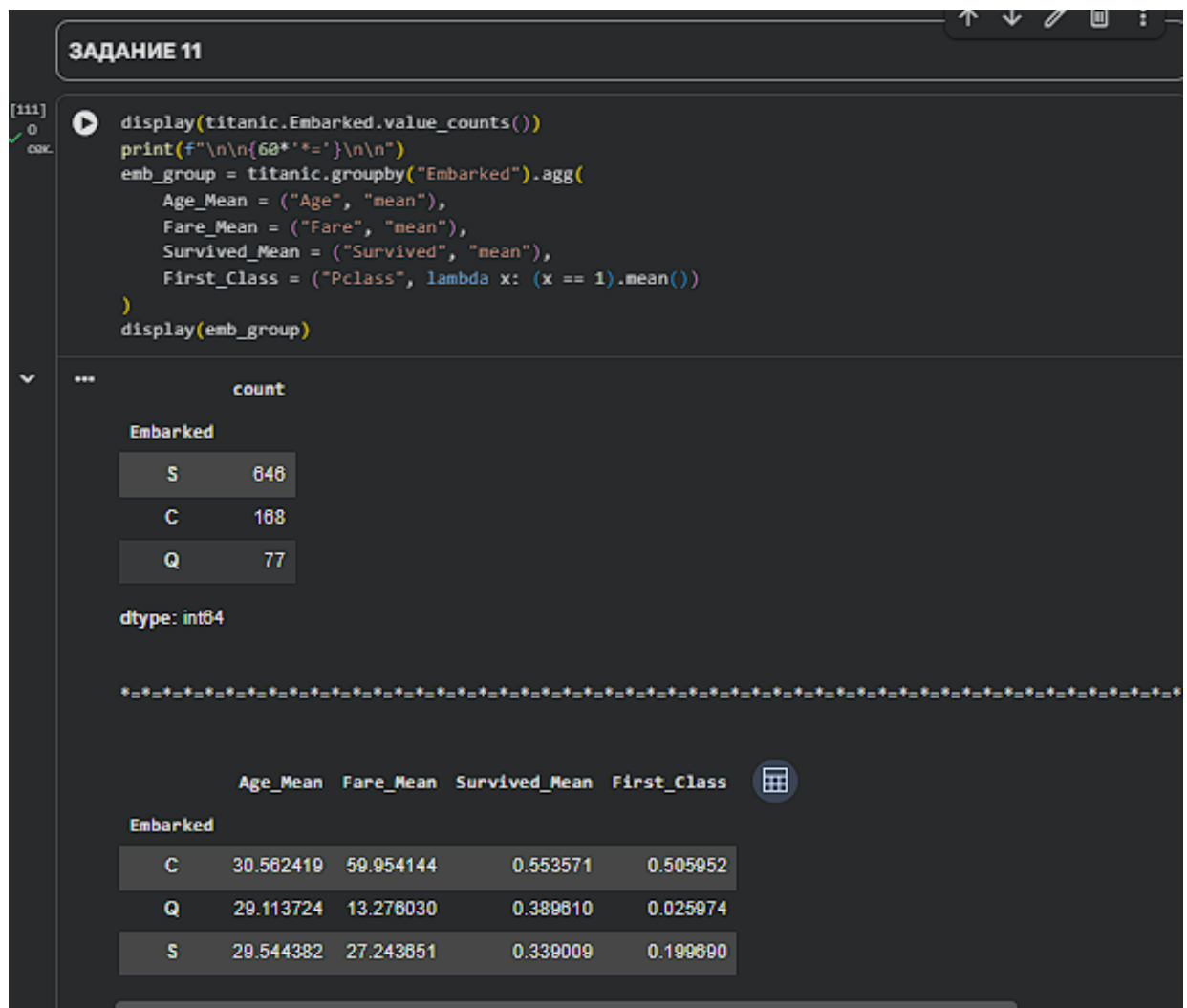


Рисунок 14. Аналитическая группировка и статистика по порту посадки.

Задание 12. Сравнительный анализ по размеру семьи.

Ход выполнения: сгруппировал данные по размеру семьи (FamilySize), вычислил статистику выживаемости, отфильтровал и отсортировал результаты для выявления закономерностей между размером семьи и шансом на спасение.

Наилучшие шансы на спасение были у пассажиров из небольших семей (2–4 человека, выживаемость до 72%). Одиночки и члены очень больших семей (более 7 человек) выживали реже всего (вплоть до 0%).

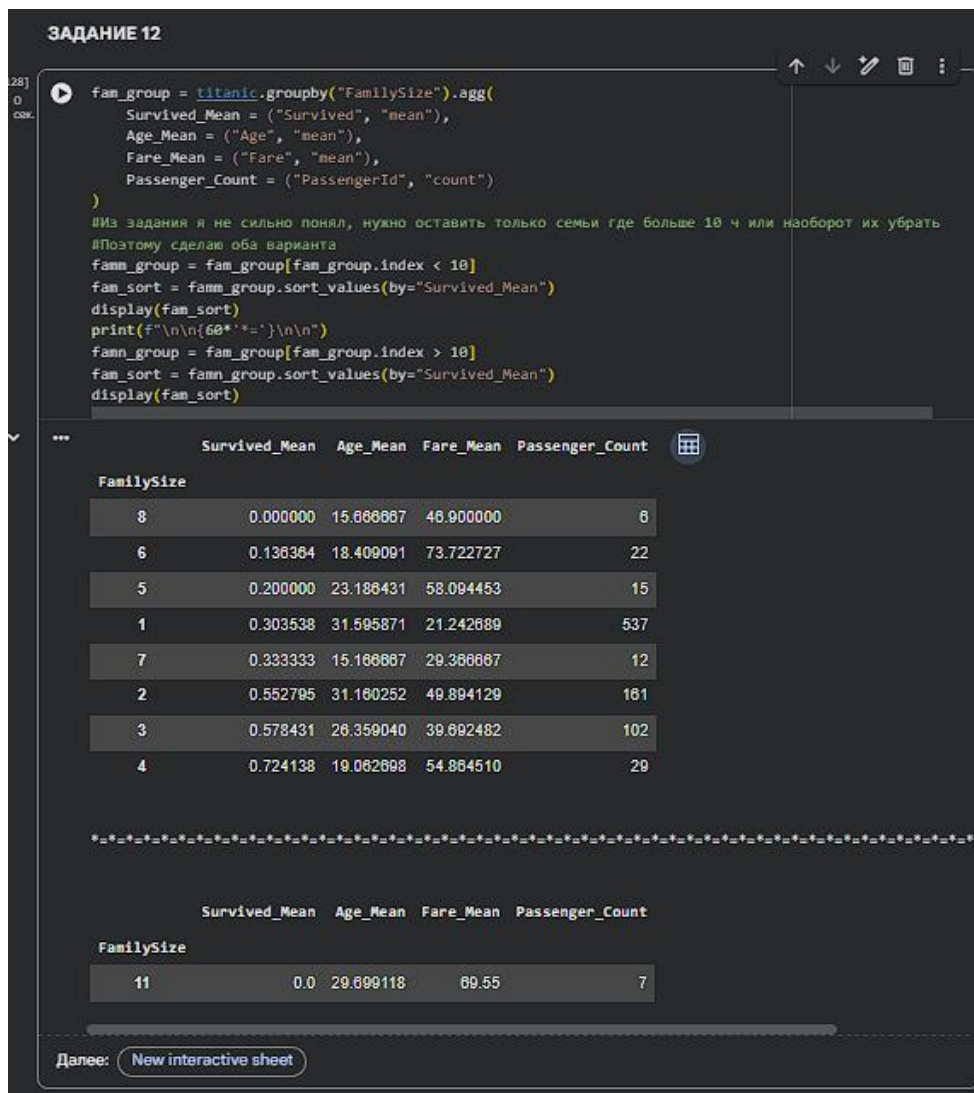


Рисунок 15. Сравнительный анализ выживаемости в зависимости от размера семьи.

Вывод:

В ходе лабораторной работы я загрузил и исследовал датасет Титаника, изучил структуру таблицы и статистические показатели. На практике освоил работу с объектами `Series` и `DataFrame`, научился обращаться к данным через `loc` и `iloc`. Выполнил фильтрацию строк по одиночным и комбинированным условиям. Сделал группировку данных с помощью `groupby` и построил сводные таблицы через `pivot_table`. Обработал пропущенные значения и удалил дубликаты. Также научился создавать новые признаки на основе существующих столбцов и разбивать непрерывные данные на интервалы методом `pd.cut`.

Блокнот этой лабораторной

Репозиторий с работами на Гитхаб

Ответы на контрольные вопросы:

1. Pandas - это библиотека Python для анализа и обработки табличных данных. Основные объекты: Series (одномерный массив) и DataFrame (двумерная таблица).
2. Series имеет метки индекса (не только числа, но и строки, даты), а обычный список - только числовые позиции. Создается так: `pd.Series(data, index=['a', 'b'])`.
3. `.loc` обращается к элементам по имени метки индекса, а `.iloc` - по числовому порядковому номеру позиции (начиная с 0).
4. Из словаря: `pd.DataFrame({'col1': [1, 2], 'col2': [3, 4]})`. Названия колонок и индекс задаются параметрами `columns` и `index` при создании.
5. Параметры `read_csv`: `sep` (разделитель), `decimal`, `names`, `skiprows`, `usecols` (загружает только выбранные столбцы для экономии памяти), `index_col`, `parse_dates`.
6. Метод `.info()` выводит количество строк, названия колонок, типы данных и количество непустых значений. Параметры: `verbose`, `memory_usage`.
7. Для числовых данных `.describe()` показывает `count`, `mean`, `std`, `min`, `max` и квартили. Для текстовых: `count`, `unique`, `top` (самое частое значение) и `freq`.
8. Доступ к столбцу можно получить двумя способами: через квадратные скобки `df['column']` или через точку `df.column`.
9. Одно условие: `df[df['Age'] > 30]`. Несколько условий объединяются через `&` (и) или `|` (или), каждое условие берется в скобки: `df[(df['Age'] > 30) & (df['Sex'] == 'female')]`.
10. Статистические методы: `.mean()`, `.max()`, `.min()`, `.sum()`, `.std()`, `.count()`. Если применить `.mean()` к текстовым столбцам, возникнет ошибка, так как текст нельзя усреднить.
11. Метод `.groupby()` делит данные на группы по значению указанного столбца. Несколько функций применяются через `.agg()`, например: `.agg({'col1': 'mean', 'col2': 'count'})`.

12. `pivot_table` строит двумерную сводную таблицу с агрегацией по строкам и столбцам, что удобно для отчетов. `groupby` дает иерархический индекс и удобнее для промежуточных вычислений.
13. Пропуски обнаруживаются через `.isnull().sum()`. Стратегии обработки: удаление строк (`.dropna()`) или заполнение (`.fillna`) средним, медианой, модой или константой.
14. Новый признак создается присваиванием: `df['new'] = df['A'] + df['B']`. Метод `pd.cut()` разбивает непрерывные числовые данные на категорические интервалы (бины).
15. Сортировка выполняется методом `.sort_values(by='имя_столбца', ascending=False)`. Чтобы вывести только топ-N строк, используется цепочка с методом `.head(N)`.